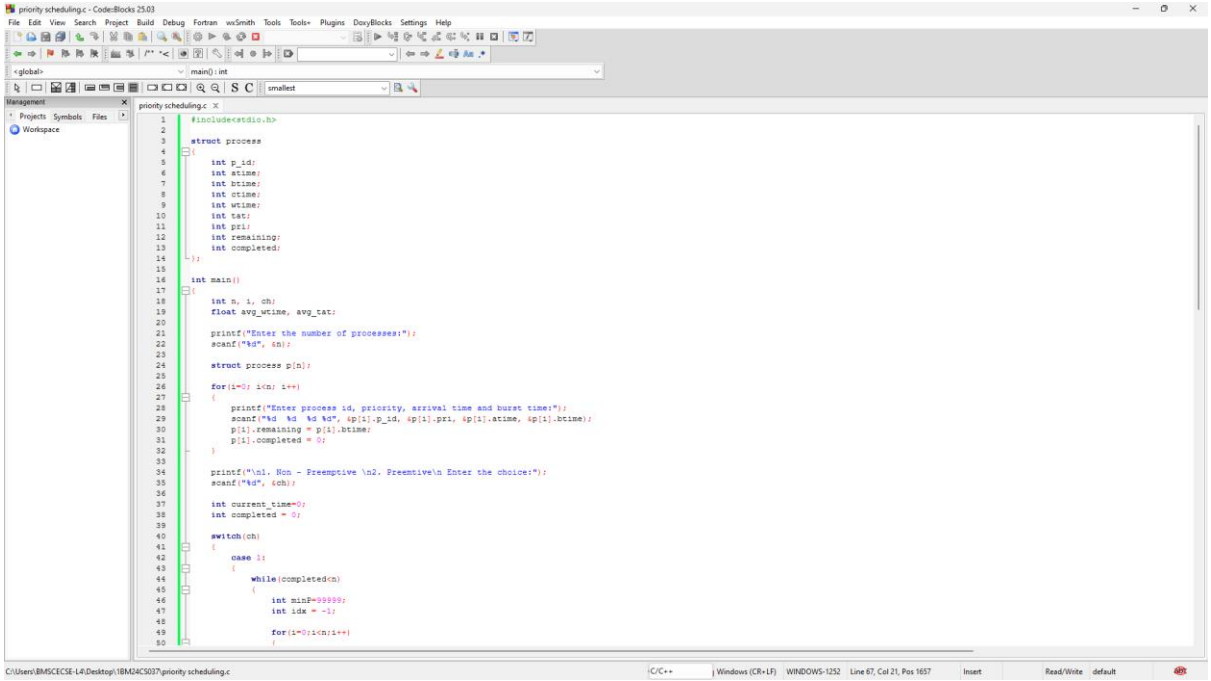
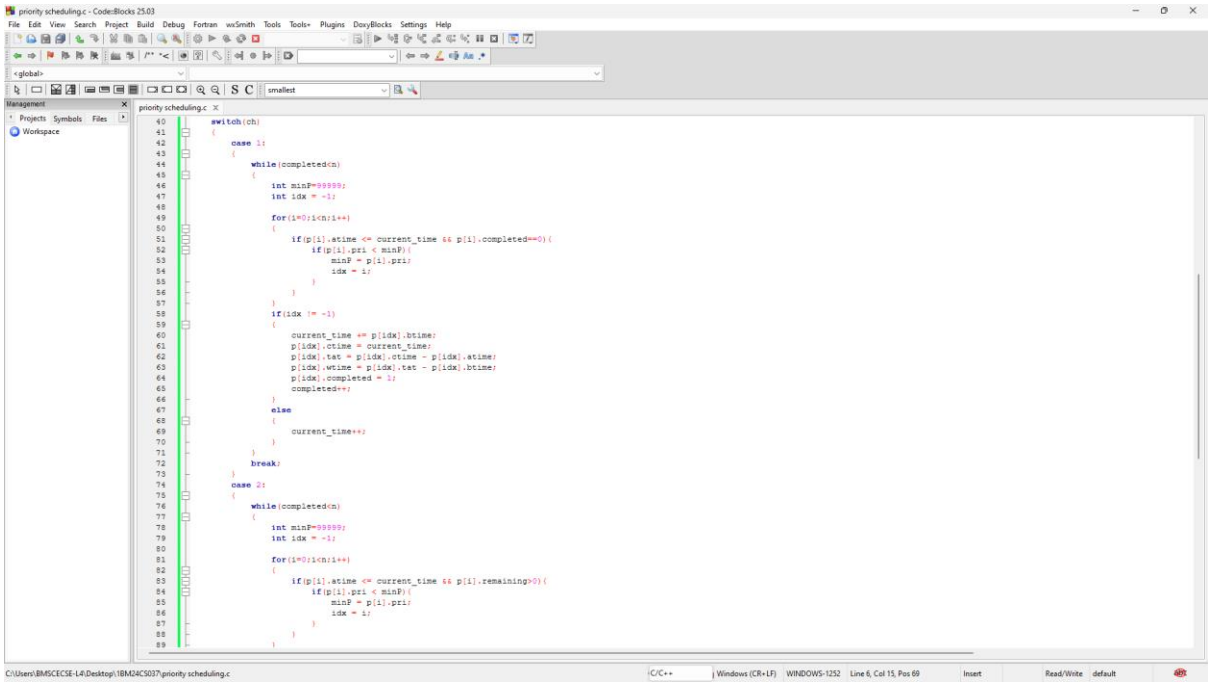


```c
#include<stdio.h>

struct process
{
    int p_id;
    int atime;
    int btime;
    int ctime;
    int wtime;
    int tat;
    int pri;
    int remaining;
    int completed;
};

int main()
{
    int n, i, ch;
    float avg_wtime, avg_tat;

    printf("Enter the number of processes:");
    scanf("%d", &n);

    struct process p[n];

    for(i=0; i<n; i++)
    {
        printf("Enter process id, priority, arrival time and burst time:");
        scanf("%d  %d  %d %d", &p[i].p_id, &p[i].pri, &p[i].atime, &p[i].btime);
        p[i].remaining = p[i].btime;
        p[i].completed = 0;
    }

    printf("\n1. Non - Preemptive \n2. Preemtive\n Enter the choice:");
    scanf("%d", &ch);

    int current_time=0;
    int completed = 0;

    switch(ch)
    {
        case 1:
        {
            while(completed<n)
            {
                int minP=99999;
                int idx = -1;

                for(i=0;i<n;i++)
                {
```



```c
    switch(ch)
    {
        case 1:
        {
            while(completed<n)
            {
                int minP=99999;
                int idx = -1;

                for(i=0;i<n;i++)
                {
                    if(p[i].atime <= current_time && p[i].completed==0){
                        if(p[i].pri < minP){
                            minP = p[i].pri;
                            idx = i;
                        }
                    }
                }
                if(idx != -1)
                {
                    current_time += p[idx].btime;
                    p[idx].ctime = current_time;
                    p[idx].tat = p[idx].ctime - p[idx].atime;
                    p[idx].wtime = p[idx].tat - p[idx].btime;
                    p[idx].completed = 1;
                    completed++;
                }
                else
                {
                    current_time++;
                }
            }
            break;
        }
        case 2:
        {
            while(completed<n)
            {
                int minP=99999;
                int idx = -1;

                for(i=0;i<n;i++)
                {
                    if(p[i].atime <= current_time && p[i].remaining>0){
                        if(p[i].pri < minP){
                            minP = p[i].pri;
                            idx = i;
                        }
                    }
                }
```

Program 3: Priority Scheduling (Non-Preemptive and Preemptive)

priority scheduling.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> | main() : int | smallest

Management: Projects Symbols Files — Workspace

priority scheduling.c

```c
                    }
                }
            }
            if(idx != -1)
            {
                p[idx].remaining--;
                current_time++;

                if(p[idx].remaining==0)
                {
                    p[idx].ctime = current_time;
                    p[idx].tat = p[idx].ctime - p[idx].atime;
                    p[idx].wtime = p[idx].tat - p[idx].btime;
                    completed++;
                }
            }
            else
            {
                current_time++;
            }
        }
        break;
    }
    default: printf("Invalid choice!");
            return 0;
}

float sum_wtime = 0;
float sum_tat = 0;

for(i=0; i<n; i++)
{
    sum_wtime += p[i].wtime;
    sum_tat += p[i].tat;
}

avg_wtime = (sum_wtime) / n;
avg_tat = (sum_tat) / n;

printf("Process ID | Priority | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time\n");
for(i=0; i<n; i++)
{
    printf("%d | %d | %d | %d | %d | %d | %d\n", p[i].p_id, p[i].pri, p[i].atime, p[i].btime, p[i].ctime, p[i].wtime, p[i].tat);
}

printf("\nAverage Waiting Time: %f ;\n Average Turnaround Time: %f", avg_wtime, avg_tat);
return 0;
}
```

C:\Users\BMSCECSE-L4\Desktop\1BM24CS037\priority scheduling.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 66, Col 18, Pos 1635 | Insert | Read/Write | default

"C:\Users\BMSCECSE-L4\Desk

```
Enter the number of processes:3
Enter process id, priority, arrival time and burst time:1 3 0 10
Enter process id, priority, arrival time and burst time:2 2 1 5
Enter process id, priority, arrival time and burst time:3 1 2 2

1. Non - Preemptive
2. Preemtive
 Enter the choice:1
Process ID | Priority | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time
1 | 3 | 0 | 10 | 10 | 0 | 10
2 | 2 | 1 | 5 | 17 | 11 | 16
3 | 1 | 2 | 2 | 12 | 8 | 10

Average Waiting Time: 6.333333 ;
 Average Turnaround Time: 12.000000
Process returned 0 (0x0)   execution time : 14.887 s
Press any key to continue.
```

```
"C:\Users\BMSCECSE-L4\Desk

Enter the number of processes:3
Enter process id, priority, arrival time and burst time:1 3 0 10
Enter process id, priority, arrival time and burst time:2 2 1 5
Enter process id, priority, arrival time and burst time:3 1 2 2

1. Non - Preemptive
2. Preemtive
 Enter the choice:2
Process ID | Priority | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time
1 | 3 | 0 | 10 | 17 | 7 | 17
2 | 2 | 1 | 5 | 8 | 2 | 7
3 | 1 | 2 | 2 | 4 | 0 | 2

Average Waiting Time: 3.000000 ;
 Average Turnaround Time: 8.666667
Process returned 0 (0x0)   execution time : 15.059 s
Press any key to continue.
```